one's complement A notation that represents the most negative value by 10 … 000_{two} and the most positive value by 01 … 11_{two}, leaving an equal number of negatives and positives but ending up with two zeros, one positive (00 … 00_{two}) and one negative (11 … 11_{two}). The term is also used to mean the inversion of every bit in a pattern: 0 to 1 and 1 to 0.

biased notation A notation that represents the most negative value by 00 … 000_{two} and the most positive value by 11 … 11_{two}, with 0 typically having the value 10 … 00_{two}, thereby biasing the number such that the number plus the bias has a non-negative representation.

A third alternative representation to two's complement and sign and magnitude is called **one's complement**. The negative of a one's complement is found by inverting each bit, from 0 to 1 and from 1 to 0, or $\overline{x}$. This relation helps explain its name since the complement of x is $2^n - x - 1$. It was also an attempt to be a better solution than sign and magnitude, and several early scientific computers did use the notation. This representation is similar to two's complement except that it also has two 0s: 00 … 00_{two} is positive 0 and 11 … 11_{two} is negative 0. The most negative number, 10 … 000_{two}, represents $-2{,}147{,}483{,}647_{ten}$, and so the positives and negatives are balanced. One's complement adders did need an extra step to subtract a number, and hence two's complement dominates today.

A final notation, which we will look at when we discuss floating point in Chapter 3, is to represent the most negative value by 00 … 000_{two} and the most positive value by 11 … 11_{two}, with 0 typically having the value 10 … 00_{two}. This representation is called a **biased notation**, since it biases the number such that the number plus the bias has a non-negative representation.

2.5 Representing Instructions in the Computer

We are now ready to explain the difference between the way humans instruct computers and the way computers see instructions.

Instructions are kept in the computer as a series of high and low electronic signals and may be represented as numbers. In fact, each piece of an instruction can be considered as an individual number, and placing these numbers side by side forms the instruction. The 32 registers of LEGv8 are just referred to by their number, from 0 to 31.

EXAMPLE

Translating a LEGv8 Assembly Instruction into a Machine Instruction

Let's do the next step in the refinement of the LEGv8 language as an example. We'll show the real LEGv8 language version of the instruction represented symbolically as

```
ADD X9,X20,X21
```

first as a combination of decimal numbers and then of binary numbers.

ANSWER

The decimal representation is

1112	21	0	20	9

Each of these segments of an instruction is called a *field*. The first field (containing 1112 in this case) tells the LEGv8 computer that this instruction performs addition. The second field gives the number of the register that is the second source operand of the addition operation (21 for `X21`), and the fourth field gives the other source operand for the addition (20 for `X20`). The fifth field contains the number of the register that is to receive the sum (9 for `X9`). (The third field is unused in this instruction, so it is set to 0.) Thus, this instruction adds register `X20` to register `X21` and places the sum in register `X9`.

This instruction can also be represented as fields of binary numbers instead of decimal:

10001011000	10101	000000	10100	01001
11 bits	5 bits	6 bits	5 bits	5 bits

This layout of the instruction is called the **instruction format**. As you can see from counting the number of bits, this LEGv8 instruction takes exactly 32 bits—a word, or one half of a doubleword. In keeping with our design principle that simplicity favors regularity, all LEGv8 instructions are 32 bits long.

instruction format A form of representation of an instruction composed of fields of binary numbers.

To distinguish it from assembly language, we call the numeric version of instructions **machine language** and a sequence of such instructions *machine code*.

machine language Binary representation used for communication within a computer system.

It would appear that you would now be reading and writing long, tiresome strings of binary numbers. We avoid that tedium by using a higher base than binary that converts easily into binary. Since almost all computer data sizes are multiples of 4, **hexadecimal** (base 16) numbers are popular. As base 16 is a power of 2, we can trivially convert by replacing each group of four binary digits by a single hexadecimal digit, and vice versa. Figure 2.4 converts between hexadecimal and binary.

hexadecimal Numbers in base 16.

Hexadecimal	Binary	Hexadecimal	Binary	Hexadecimal	Binary	Hexadecimal	Binary
0_{hex}	0000_{two}	4_{hex}	0100_{two}	8_{hex}	1000_{two}	c_{hex}	1100_{two}
1_{hex}	0001_{two}	5_{hex}	0101_{two}	9_{hex}	1001_{two}	d_{hex}	1101_{two}
2_{hex}	0010_{two}	6_{hex}	0110_{two}	a_{hex}	1010_{two}	e_{hex}	1110_{two}
3_{hex}	0011_{two}	7_{hex}	0111_{two}	b_{hex}	1011_{two}	f_{hex}	1111_{two}

FIGURE 2.4 The hexadecimal–binary conversion table. Just replace one hexadecimal digit by the corresponding four binary digits, and vice versa. If the length of the binary number is not a multiple of 4, go from right to left.

Because we frequently deal with different number bases, to avoid confusion, we will subscript decimal numbers with *ten*, binary numbers with *two*, and hexadecimal numbers with *hex*. (If there is no subscript, the default is base 10.) By the way, C and Java use the notation 0x*nnnn* for hexadecimal numbers.

EXAMPLE

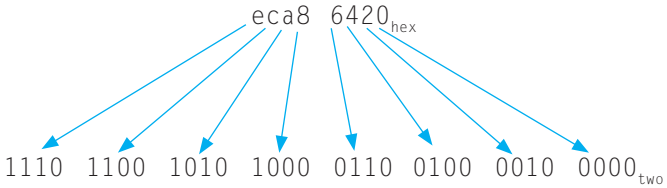
Binary to Hexadecimal and Back

Convert the following 8-digit hexadecimal and 32-bit binary numbers into the other base:

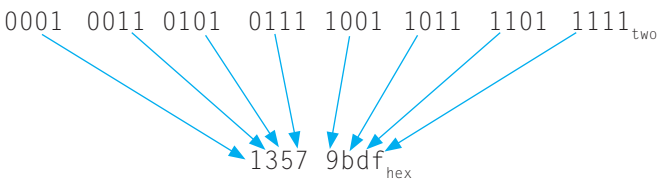
$$\texttt{eca8 6420}_{\text{hex}}$$
$$\texttt{0001 0011 0101 0111 1001 1011 1101 1111}_{\text{two}}$$

ANSWER

Using Figure 2.4, the answer is just a table lookup one way:



And then the other direction:



LEGv8 Fields

LEGv8 fields are given names to make them easier to discuss:

opcode	Rm	shamt	Rn	Rd
11 bits	5 bits	6 bits	5 bits	5 bits

Here is the meaning of each name of the fields in LEGv8 instructions:

opcode The field that denotes the operation and format of an instruction.

- ***opcode***: Basic operation of the instruction, and this abbreviation is its traditional name.
- *Rm:* The second register source operand.
- *shamt:* Shift amount. (Section 2.6 explains shift instructions and this term; it will not be used until then, and hence the field contains zero in this section.)
- *Rn:* The first register source operand.
- *Rd:* The register destination operand. It gets the result of the operation.

A problem occurs when an instruction needs longer fields than those shown above. For example, the load register instruction must specify two registers and a constant. If the address were to use one of the 5-bit fields in the format above, the largest constant within the load register instruction would be limited to only 2^5-1 or 31. This constant is used to select elements from arrays or data structures, and it often needs to be much larger than 31. This 5-bit field is too small to be useful.

Hence, we have a conflict between the desire to keep all instructions the same length and the desire to have a single instruction format. This conflict leads us to the final hardware design principle:

Design Principle 3: Good design demands good compromises.

The compromise chosen by the LEGv8 designers is to keep all instructions the same length, thereby requiring distinct instruction formats for different kinds of instructions. For example, the format above is called *R-type* (for register) or *R-format*. A second type of instruction format is *D-type* or *D-format* and is used by the data transfer instructions (loads and stores). The fields of D-format are

opcode	address	op2	Rn	Rt
11 bits	9 bits	2 bits	5 bits	5 bits

The 9-bit address means a load register instruction can load any doubleword within a region of $\pm 2^8$ or 256 bytes ($\pm 2^5$ or 32 doublewords) of the address in the base register Rn. We see that more than 32 registers would be difficult in this format, as the Rn and Rt fields would each need another bit, making it harder to fit everything in one word. (The last field of D-type is called Rt instead of Rd because for store instructions, the field indicates a data source and not a data destination.)

Let's look at the load register instruction from page 72:

```
LDUR X9, [X22,#64] // Temporary reg X9 gets A[8]
```

Here, 22 (for `X22`) is placed in the Rn field, 64 is placed in the address field, and 9 (for `X9`) is placed in the Rt field. Note that in a load register instruction, the Rt field specifies the *destination* register, which receives the result of the load.

We also need a format for the immediate instructions `ADDI`, `SUBI`, and immediate instructions that we will introduce later. While we could have used the D-format instruction since it has a 9-bit field holding a constant, the ARMv8 architects decided it would be useful to have a larger immediate field for these instructions, even shaving a bit from the opcode field to make a 12-bit immediate. The fields of *immediate* or *I-type* format are

opcode	immediate	Rn	Rd
10 bits	12 bits	5 bits	5 bits

Although multiple formats complicate the hardware, we can reduce the complexity by keeping the formats similar. For example, the last two fields of all three formats are the identical size and almost the same names, and the opcode field is the same size in two of the three formats.

In case you were wondering, the formats are distinguished by the values in the first field: each format is assigned a distinct set of values in the first field (opcode) so that the hardware knows how to treat the rest of the instruction. Figure 2.5 shows the numbers used in each field for the LEGv8 instructions covered so far.

Instruction	Format	opcode	Rm	shamt	address	op2	Rn	Rd
ADD (add)	R	1112_{ten}	reg	0	n.a.	n.a.	reg	reg
SUB (subtract)	R	1624_{ten}	reg	0	n.a.	n.a.	reg	reg
ADDI (add immediate)	I	580_{ten}	reg	n.a.	constant	n.a.	reg	n.a.
SUBI (sub immediate)	I	836_{ten}	reg	n.a.	constant	n.a.	reg	n.a.
LDUR (load word)	D	1986_{ten}	reg	n.a.	address	0	reg	n.a.
STUR (store word)	D	1984_{ten}	reg	n.a.	address	0	reg	n.a.

FIGURE 2.5 LEGv8 instruction encoding. In the table above, "reg" means a register number between 0 and 31, "address" means a 9-bit address or 12-bit constant, and "n.a." (not applicable) means this field does not appear in this format. The op2 field expands the opcode field.

EXAMPLE

Translating LEGv8 Assembly Language into Machine Language

We can now take an example all the way from what the programmer writes to what the computer executes. If `X10` has the base of the array `A` and `X21` corresponds to `h`, the assignment statement

```
A[30] = h + A[30] + 1;
```

is compiled into

```
LDUR X9, [X10,#240] // Temporary reg X9 gets A[30]
ADD  X9,X21,X9      // Temporary reg X9 gets h+A[30]
ADDI X9,X9,#1       // Temporary reg X9 gets h+A[30]+1
STUR X9, [X10,#240] // Stores h+A[30]+1 back into A[30]
```

What is the LEGv8 machine language code for these three instructions?

ANSWER

For convenience, let's first represent the machine language instructions using decimal numbers. From Figure 2.5, we can determine the three machine language instructions:

opcode	Rm/address	shamt/op2	Rn	Rd/Rt
1986	240	0	10	9
1112	9	0	21	9
580	1		9	9
1984	240	0	10	9

The LDUR instruction is identified by 1986 (see Figure 2.5) in the first field (opcode). The base register 10 is specified in the fourth field (Rn), and the destination register 9 is specified in the last field (Rt). The offset to select A[30] ($240 = 30 \times 8$) is found in the second field (address).

The ADD instruction that follows is specified with 1112 in the first field (opcode). The three register operands (9, 21, and 9) are found in the second, fourth, and fifth fields, with 0 in the third field (shamt).

The following ADDI instruction is specified with 580 in the first field (opcode), the immediate value 1 in the second, and the register operands (9 in both cases) in the last two fields.

The STUR instruction is identified with 1984 in the first field. The rest of this final instruction is identical to the LDUR instruction.

Since $240_{ten} = 0\ 1111\ 0000_{two}$, the binary equivalent to the decimal form is:

1111100001	0	011110000	00	01010	01001
10001011000		01001	000000	10101	01001
1001000100		000000000001		01001	01001
1111100000	0	011110000	00	01010	01001

Note the similarity of the binary representations of the first and last instructions. The only difference is in the tenth bit from the left, which is highlighted here.

Elaboration: ARMv8 assembly language programmers aren't forced to use ADDI when working with constants. The programmer simply writes ADD, and the assembler generates the proper opcode and the proper instruction format depending on whether the operands are all registers (R-format) or if one is a constant (I-format). We use the explicit names in LEGv8 for the different opcodes and formats as we think it is less confusing when introducing assembly language versus machine language.

Elaboration: Note that unlike MIPS, the LEGv8 immediate field in I-format is zero-extended. Thus, LEGv8 includes both ADDI and SUBI instructions, while MIPS has just ADDI and both positive and negative immediates.

Hardware/Software Interface

The desire to keep all instructions the same size conflicts with the desire to have as many registers as possible. Any increase in the number of registers uses up at least one more bit in every register field of the instruction format. Given these constraints and the design principle that smaller is faster, most instruction sets today have 16 or 32 general-purpose registers.

Figure 2.6 summarizes the portions of LEGv8 machine language described in this section. As we shall see in Chapter 4, the similarity of the binary representations of related instructions simplifies hardware design. These similarities are another example of regularity in the LEGv8 architecture.

LEGv8

Name	Format	Example						Comments
ADD	R	1112	3	0		2	1	ADD X1, X2, X3
SUB	R	1624	3	0		2	1	SUB X1, X2, X3
ADDI	I	580	100			2	1	ADDI X1, X2, #100
SUBI	I	836	100			2	1	SUBI X1, X2, #100
LDUR	D	1986	100		0	2	1	LDUR X1, [X2, #100]
STUR	D	1984	100		0	2	1	STUR X1, [X2, #100]
Field size		11 or 10 bits	5 bits	5 or 4 bits	2 bits	5 bits	5 bits	All ARM instructions are 32 bits long
R-format	R	opcode	Rm	shamt		Rn	Rd	Arithmetic instruction format
I-format	I	opcode	immediate			Rn	Rd	Immediate format
D-format	D	opcode	address		op2	Rn	Rt	Data transfer format

FIGURE 2.6 LEGv8 architecture revealed through Section 2.5. The three LEGv8 instruction formats so far are R, I and D. The last 10 bits contain a *Rn* field, giving one of the sources; and the *Rd* or *Rt* field, which specifies the destination register, except for store register, where it specifies the value to be stored. R-format divides the rest into an 11-bit opcode; a 5-bit *Rm* field, specifying the other source operand; and a 6-bit *shamt* field, which Section 2.6 explains. I-format combines 12 bits into a single *immediate* field, which requires shrinking the opcode field to 10 bits. The D-format uses a full 11-bit opcode like the R-format, plus a 9-bit *address* field, and a 2-bit *op2* field. The op2 field is logically an extension of the opcode field.

The BIG Picture

Today's computers are built on two key principles:

1. Instructions are represented as numbers.
2. Programs are stored in memory to be read or written, just like data.

These principles lead to the *stored-program* concept; its invention let the computing genie out of its bottle. Figure 2.7 shows the power of the concept; specifically, memory can contain the source code for an editor program, the corresponding compiled machine code, the text that the compiled program is using, and even the compiler that generated the machine code.

One consequence of instructions as numbers is that programs are often shipped as files of binary numbers. The commercial implication is that computers can inherit ready-made software provided they are compatible with an existing instruction set. Such "binary compatibility" often leads industry to align around a small number of instruction set architectures.

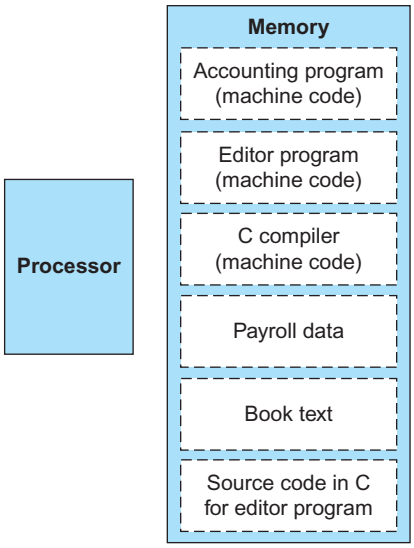


FIGURE 2.7 The stored-program concept. Stored programs allow a computer that performs accounting to become, in the blink of an eye, a computer that helps an author write a book. The switch happens simply by loading memory with programs and data and then telling the computer to begin executing at a given location in memory. Treating instructions in the same way as data greatly simplifies both the memory hardware and the software of computer systems. Specifically, the memory technology needed for data can also be used for programs, and programs like compilers, for instance, can translate code written in a notation far more convenient for humans into code that the computer can understand.

Check Yourself

What LEGv8 instruction does this represent? Choose from one of the four options below.

opcode	Rm	shamt	Rn	Rd
1624	9	0	10	11

1. `SUB X9, X10, X11`
2. `ADD X11, X9, X10`
3. `SUB X11, X10, X9`
4. `SUB X11, X9, X10`

Elaboration: You might be asking yourself why the LEGv8 opcode field is so big given the modest number of instructions in Figure 2.1? The main reason is that the full ARMv8 instruction set is very large; depending how you count, it is on the order of 1000 instructions. We'll survey the full ARMv8 instruction set in some of the last sections of this chapter and Chapters 3 and 5.

2.6 Logical Operations

"Contrariwise," continued Tweedledee, "if it was so, it might be; and if it were so, it would be; but as it isn't, it ain't. That's logic."

Lewis Carroll, *Alice's Adventures in Wonderland*, 1865

Although the first computers operated on full words, it soon became clear that it was useful to operate on fields of bits within a word or even on individual bits. Examining characters within a word, each of which is stored as 8 bits, is one example of such an operation (see Section 2.9). It follows that operations were added to programming languages and instruction set architectures to simplify, among other things, the packing and unpacking of bits into words. These instructions are called *logical operations*. Figure 2.8 shows logical operations in C, Java, and LEGv8.

Logical operations	C operators	Java operators	LEGv8 instructions
Shift left	<<	<<	LSL
Shift right	>>	>>>	LSR
Bit-by-bit AND	&	&	AND,ANDI
Bit-by-bit OR	|	|	ORR,ORRI
Bit-by-bit NOT	~	~	EOR,EORI

FIGURE 2.8 C and Java logical operators and their corresponding LEGv8 instructions. One way to implement NOT is to use EOR with one operand being all ones (FFFF FFFF FFFF $FFFF_{hex}$).

The first class of such operations is called *shifts*. They move all the bits in a doubleword to the left or right, filling the emptied bits with 0s. For example, if register X19 contained

$$00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00001001_{two} = 9_{ten}$$

and the instruction to shift left by 4 was executed, the new value would be:

$$00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 10010000_{two} = 144_{ten}$$

The dual of a shift left is a shift right. The actual names of the two LEGv8 shift instructions are *logical shift left* (LSL) and *logical shift right* (LSR). The following instruction performs the operation above, if the original value was in register X19 and the result should go in register X11:

```
LSL X11,X19,#4 // reg X11 = reg X19 << 4 bits
```

We delayed explaining the *shamt* field in the R-format. Used in shift instructions, it stands for *shift amount*. Hence, the machine language version of the instruction above is:

opcode	Rm	shamt	Rn	Rd
1691	0	4	19	11

The encoding of LSL is 1691 in the opcode field, Rd contains 11, Rn contains 19, and shamt contains 4. The Rm field is unused and thus is set to 0.

Shift left logical provides a bonus benefit. Shifting left by i bits gives the identical result as multiplying by 2^i, just as shifting a decimal number by i digits is equivalent to multiplying by 10^i. For example, the above LSL shifts by 4, which gives the same result as multiplying by 2^4 or 16. The first bit pattern above represents 9, and $9 \times 16 = 144$, the value of the second bit pattern.

Another useful operation that isolates fields is **AND**. (We capitalize the word to avoid confusion between the operation and the English conjunction.) AND is a bit-by-bit operation that leaves a 1 in the result only if both bits of the operands are 1. For example, if register X11 contains

AND A logical bit-by-bit operation with two operands that calculates a 1 only if there is a 1 in *both* operands.

```
00000000 00000000 00000000 00000000 00000000 00000000 00001101 11000000two
```

and register X10 contains

```
00000000 00000000 00000000 00000000 00000000 00000000 00111100 00000000two
```

then, after executing the LEGv8 instruction

```
AND X9,X10,X11      // reg X9 = reg X10 & reg X11
```

the value of register X9 would be

```
00000000 00000000 00000000 00000000 00000000 00000000 00001100 00000000two
```

As you can see, AND can apply a bit pattern to a set of bits to force 0s where there is a 0 in the bit pattern. Such a bit pattern in conjunction with AND is traditionally called a *mask*, since the mask "conceals" some bits.

To place a value into one of these seas of 0s, there is the dual to AND, called **OR**. It is a bit-by-bit operation that places a 1 in the result if *either* operand bit is a 1. To elaborate, if the registers X10 and X11 are unchanged from the preceding example, the result of the LEGv8 instruction

OR A logical bit-by-bit operation with two operands that calculates a 1 if there is a 1 in *either* operand.

```
ORR X9,X10,X11 // reg X9 = reg X10 | reg X11
```

is this value in register X9:

```
00000000 00000000 00000000 00000000 00000000 00000000 00111101 11000000two
```

The final logical operation is a contrarian. **NOT** takes one operand and places a 1 in the result if one operand bit is a 0, and vice versa. Using our prior notation, it calculates $\overline{x}$.

NOT A logical bit-by-bit operation with one operand that inverts the bits; that is, it replaces every 1 with a 0, and every 0 with a 1.

EOR A logical bit-by-bit operation with two operands that calculates the Exclusive OR of the two operands. That is, it calculates a 1 only if the values are different in the two operands.

In keeping with the three-operand format, the designers of ARMv8 decided to include the instruction **EOR** (Exclusive OR) instead of NOT. Since exclusive OR creates a 0 when bits are the same and a 1 if they are different, the equivalent to NOT is an EOR 111…111.

If the register `X10` is unchanged from the preceding example and register `X12` has the value 0, the result of the LEGv8 instruction

```
EOR X9,X10,X12 // reg X9 = reg X10 | reg X12
```

is this value in register `X9`:

```
00000000 00000000 00000000 00000000 00000000 00000000 00110001 11000000two
```

Figure 2.8 above shows the relationship between the C and Java operators and the LEGv8 instructions. Constants are useful in logical operations as well as in arithmetic operations, so LEGv8 also provides the instructions *and immediate* (`ANDI`), *or immediate* (`ORRI`), and *exclusive or immediate* (`EORI`).

Elaboration: C allows *bit fields* or *fields* to be defined within doublewords, both allowing objects to be packed within a doubleword and to match an externally enforced interface such as an I/O device. All fields must fit within a single doubleword. Fields are unsigned integers that can be as short as 1 bit. C compilers insert and extract fields using logical instructions in LEGv8: `AND`, `ORR`, `LSL`, and `LSR`.

Elaboration: The immediate fields for `ANDI`, `ORRI`, and `EORI` of the full ARMv8 instruction set are not simple 12-bit immediates. Once again, like ARMv7, it has the unusual feature of using a complex algorithm for encoding immediate values; ARMv8 does it with repeating patterns. This means that some small constants (e.g., 1, 2, 3, 4, and 6) are valid, while others (e.g., –1, 0, 5) are not. LEGv8 simply uses normal 12-bit immediates as found in `ADDI`. This difference means `EORI X1,X1,#5` is legal for LEGv8 but not ARMv8. Once again, in addition to its rarity among other instruction sets, immediate encoding is omitted because it would complicate the datapaths in Chapter 4 significantly.

Check Yourself

Which operations can isolate a field in a doubleword?

1. AND
2. A shift left followed by a shift right

Elaboration: Unlike almost all other computer architectures, ARMv8 (and ARMv7) allows a register to be shifted as part of an arithmetic or logical instruction: add an optionally shifted register, subtract an optionally shifted register, AND an optionally shifted register, and so on. Since this combination is unusual in computer architectures and not frequently generated by compilers—and since supporting it would make the data path in Chapter 4 much more complicated and unlike other computer datapaths—we decided to treat shifts as separate instructions, as it is in virtually every other computer architecture. While you can synthesize a shift using either `ADD` with `XZR` or `OR` with `XZR`, it would be confusing to use the same opcode for shifts as `ADD` or `OR`. Thus, we follow the ARMv8 recommendation of using an `UBFM` (unsigned bitfield move) instruction and its opcode. We simplify the values put into the Rm and shamt fields to be 0 and the actual immediate shift amount, which is what it looks like in ARMv8 assembly language. The actual values in the Rm and shamt fields of `UBFM` should be (–shift amount MOD 64) and (63 – shift amount) for `LSL` and shift amount and 63 for `LSR`. The opcode field includes part of immediate field in ARMv8, so we make the two opcodes 1691 and 1690, respectively, to distinguish them.

2.7 Instructions for Making Decisions

What distinguishes a computer from a simple calculator is its ability to make decisions. Based on the input data and the values created during computation, different instructions execute. Decision making is commonly represented in programming languages using the *if* statement, sometimes combined with *go to* statements and labels. LEGv8 assembly language includes two decision-making instructions, similar to an *if* statement with a *go to*. The first instruction is

```
CBZ register, L1
```

This instruction means go to the statement labeled `L1` if the value in `register` equals zero. The mnemonic `CBZ` stands for *compare and branch if zero*. The second instruction is

```
CBNZ register, L1
```

It means go to the statement labeled `L1` if the value in `register` does *not* equal zero. The mnemonic `CBNZ` stands for *compare and branch if not zero*. These two instructions are traditionally called **conditional branches**.

The utility of an automatic computer lies in the possibility of using a given sequence of instructions repeatedly, the number of times it is iterated being dependent upon the results of the computation.... This choice can be made to depend upon the sign of a number (zero being reckoned as plus for machine purposes). Consequently, we introduce an [instruction] (the conditional transfer [instruction]) which will, depending on the sign of a given number, cause the proper one of two routines to be executed.

Burks, Goldstine, and von Neumann, 1947

EXAMPLE

Compiling *if-then-else* into Conditional Branches

In the following code segment, `f`, `g`, `h`, `i`, and `j` are variables. If the five variables `f` through `j` correspond to the five registers `X19` through `X23`, what is the compiled LEGv8 code for this C *if* statement?

```
if (i == j) f = g + h; else f = g – h;
```

ANSWER

Figure 2.9 shows a flowchart of what the LEGv8 code should do. The first expression compares for equality between two variables in registers. Given that the instructions above can only test to see if a register is zero, the first step is to subtract `j` from `i` to test if the difference is zero. It would seem that we would next want to branch if the difference is zero (`CBZ`). In general, the code will be more efficient if we test for the opposite condition to branch over the code that branches if the difference is *not* equal to zero (`CBNZ`). Here are the two instructions, using register `X9` to hold the result of subtracting `j` from `i`:

conditional branch An instruction that tests a value and that allows for a subsequent transfer of control to a new address in the program based on the outcome of the test.

```
SUB  X9,X22,X23 // X9 = i – j
CBNZ X9, Else   // go to Else if i ≠ j (X9 ≠ 0)
```

The next assignment statement performs a single operation, and if all the operands are allocated to registers, it is just one instruction:

```
ADD X19,X20,X21       // f = g + h (skipped if i ≠ j)
```

We now need to go to the end of the *if* statement. This example introduces another kind of branch, often called an *unconditional branch*. This instruction says that the processor always follows the branch. To distinguish between conditional and unconditional branches, the LEGv8 name for this type of instruction is *branch*, abbreviated as `B` (the label `Exit` is defined below).

```
B Exit       // go to Exit
```

The assignment statement in the *else* portion of the *if* statement can again be compiled into a single instruction. We just need to append the label `Else` to this instruction. We also show the label `Exit` that is after this instruction, showing the end of the *if-then-else* compiled code:

```
Else:SUB X19,X20,X21    // f = g – h (skipped if i = j)
Exit:
```

Notice that the assembler relieves the compiler and the assembly language programmer from the tedium of calculating addresses for branches, just as it does for calculating data addresses for loads and stores (see Section 2.12).

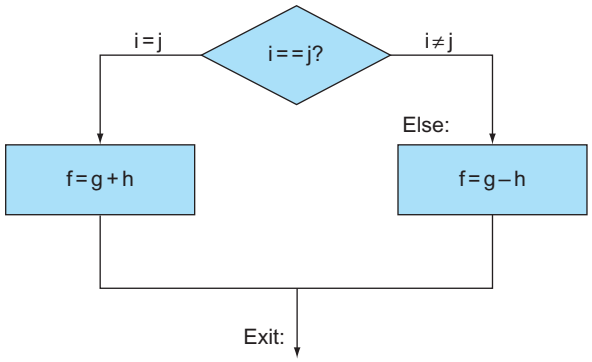


FIGURE 2.9 Illustration of the options in the *if* statement above. The left box corresponds to the *then* part of the *if* statement, and the right box corresponds to the *else* part.

Hardware/ Software Interface

Compilers frequently create branches and labels where they do not appear in the programming language. Avoiding the burden of writing explicit labels and branches is one benefit of writing in high-level programming languages and is a reason coding is faster at that level.

Loops

Decisions are important both for choosing between two alternatives—found in *if* statements—and for iterating a computation—found in loops. The same assembly instructions are the building blocks for both cases.

EXAMPLE

Compiling a *while* Loop in C

Here is a traditional loop in C:

```
while (save[i] == k)
       i += 1;
```

Assume that `i` and `k` correspond to registers `X22` and `X24` and the base of the array `save` is in `X25`. What is the LEGv8 assembly code corresponding to this C code?

ANSWER

The first step is to load `save[i]` into a temporary register. Before we can load `save[i]` into a temporary register, we need to have its address. Before we can add `i` to the base of array `save` to form the address, we must multiply the index `i` by 8 due to the byte addressing issue. Fortunately, we can use shift

left, since shifting left by 3 bits multiplies by 2^3 or 8 (see page 91 in the prior section). We need to add the label `Loop` to it so that we can branch back to that instruction at the end of the loop:

```
Loop: LSL X10,X22,#3      // Temp reg X10 = i * 8
```

To get the address of `save[i]`, we need to add `X10` and the base of save in `X25`:

```
ADD X10,X10,X25        // X10 = address of save[i]
```

Now we can use that address to load `save[i]` into a temporary register:

```
LDUR X9, [X10,#0]        // Temp reg X9 = save[i]
```

The next instruction subtracts `k` from `save[i]` and puts the difference into `X11` to set up the loop test. If `X11` is not 0, then they are unequal (`save[i] ≠ k`):

```
SUB X11,X9,X24        // X11 = save[i] – k
```

The next instruction performs the loop test, exiting if `save[i] ≠ k`:

```
CBNZ X11, Exit      // go to Exit if save[i] ≠ k(X11 ≠ 0)
```

The following instruction adds 1 to `i`:

```
ADDI X22,X22,#1        // i = i + 1
```

The end of the loop branches back to the *while* test at the top of the loop. We just add the `Exit` label after it, and we're done:

```
B       Loop        // go to Loop

Exit:
```

(See the exercises for an optimization of this sequence.)

Hardware/ Software Interface

Such sequences of instructions that end in a branch are so fundamental to compiling that they are given their own buzzword: a **basic block** is a sequence of instructions without branches, except possibly at the end, and without branch targets or branch labels, except possibly at the beginning. One of the first early phases of compilation is breaking the program into basic blocks.

basic block A sequence of instructions without branches (except possibly at the end) and without branch targets or branch labels (except possibly at the beginning).

The test for equality or inequality is probably the most popular test, but there are many other relationships between two numbers. For example, a *for* loop may want to test to see if the index variable is less than 0. The full set of comparisons is less than (<), less than or equal (≤), greater than (>), greater than or equal (≥), equal (=), and not equal (≠).

Comparison of bit patterns must also deal with the dichotomy between signed and unsigned numbers. Sometimes a bit pattern with a 1 in the most significant bit represents a negative number and, of course, is less than any positive number, which must have a 0 in the most significant bit. With unsigned integers, on the other hand, a 1 in the most significant bit represents a number that is *larger* than any that begins with a 0. (We'll soon take advantage of this dual meaning of the most significant bit to reduce the cost of the array bounds checking.)

Architects long ago figured out how to handle all these cases by keeping just four extra bits that record what occurred during an instruction. These four added bits, called *condition codes* or *flags*, are named:

- *negative (N)* – the result that set the condition code had a 1 in the most significant bit;
- *zero (Z)* – the result that set the condition code was 0;
- *overflow (V)* – the result that set the condition code overflowed; and
- *carry (C)* – the result that set the condition code had a carry out of the most significant bit or a borrow into the most significant bit.

Conditional branches then use combinations of these condition codes to perform the desired sets. In LEGv8, this conditional branch instruction is `B.cond. cond` can be used for any of the signed comparison instructions: `EQ` (= or Equal), `NE` (≠ or Not Equal), `LT` (< or Less Than), `LE` (≤ or Less than or Equal), `GT` (> or Greater Than), or `GE` (≥ or Greater than or Equal). It can also be used for the unsigned comparison instruction: `LO` (< or Lower), `LS` (≤ or Lower or Same), `HI` (> or Higher), or `HS` (≥ or Higher or Same). If the instruction that set the condition codes was a subtract (A-B), Figure 2.10 shows the LEGv8 instructions and the values of the condition codes that perform the full set of comparisons for signed and unsigned numbers.

In addition to the 10 conditional branch instructions in Figure 2.10, LEGv8 includes these four branches to complete the testing of the individual condition code bits:

- Branch on minus `(B.MI)`: N= 1;
- Branch on plus `(B.PL)`: N= 0;
- Branch on overflow set `(B.VS)`: V= 1;
- Branch on overflow clear `(B.VC)`: V= 0.

One alternative to condition codes is to have instructions that compare two registers and then branch based on the result. A second option is to compare two registers and set a third register to a result indicating the success of the comparison,

	Signed numbers		Unsigned numbers	
Comparison	Instruction	CC Test	Instruction	CC Test
=	B.EQ	Z=1	B.EQ	Z=1
≠	B.NE	Z=0	B.NE	Z=0
<	B.LT	N!=V	B.LO	C=0
≤	B.LE	~(Z=0 & N=V)	B.LS	~(Z=0 & C=1)
>	B.GT	(Z=0 & N=V)	B.HI	(Z=0 & C=1)
≥	B.GE	N=V	B.HS	C=1

FIGURE 2.10 How to do all comparisons if the instruction that set the condition codes was a subtract. If it was an ADD or AND, the test is simply on the result of the operation as compared to zero. For AND, C and V are always set to 0.

which a subsequent conditional branch instruction then tests to see if register is non-zero (condition is true) or zero (condition is false). Conditional branches in MIPS follow the latter approach (see Section 2.16).

One downside to condition codes is that if many instructions always set them, it will create dependencies that will make it difficult for pipelined execution (see Chapter 4). Hence, LEGv8 limits condition code (flag) setting to just a few instructions—ADD, ADDI, AND, ANDI, SUB, and SUBI—and even then condition code setting is optional. In LEGv8 assembly language, you simply append an S to the end of one of these instructions if you want to set condition codes: ADDS, ADDIS, ANDS, ANDIS, SUBS, and SUBIS. The instruction name actually uses the term flag, so the proper name of ADDS is "add and set flags."

Bounds Check Shortcut

Treating signed numbers as if they were unsigned gives us a low-cost way of checking if $0 \le x < y$, which matches the index out-of-bounds check for arrays. The key is that negative integers in two's complement notation look like large numbers in unsigned notation; that is, the most significant bit is a sign bit in the former notation but a large part of the number in the latter. Thus, an unsigned comparison of $x < y$ checks if x is negative as well as if x is less than y.

EXAMPLE

Use this shortcut to reduce an index-out-of-bounds check: branch to IndexOutOfBounds if X20 ≥ X11 or if X20 is negative.

ANSWER

The checking code just uses unsigned greater than or equal to do both checks:

```
SUBS XZR,X20,X11 // Test if X20 >= length or X20 < 0
B.HS IndexOutOfBounds //if bad, goto Error
```

Case/Switch Statement

Most programming languages have a *case* or *switch* statement that allows the programmer to select one of many alternatives depending on a single value. The simplest way to implement *switch* is via a sequence of conditional tests, turning the *switch* statement into a chain of *if-then-else* statements.

Sometimes the alternatives may be more efficiently encoded as a table of addresses of alternative instruction sequences, called a **branch address table** or **branch table**, and the program needs only to index into the table and then branch to the appropriate sequence. The branch table is therefore just an array of double-words containing addresses that correspond to labels in the code. The program loads the appropriate entry from the branch table into a register. It then needs to branch using the address in the register. To support such situations, computers like LEGv8 include a *branch register* instruction (`BR`), meaning an unconditional branch to the address specified in a register. Then it branches to the proper address using this instruction. We'll see an even more popular use of `BR` in the next section.

branch address table Also called **branch table**. A table of addresses of alternative instruction sequences.

Hardware/Software Interface

Although there are many statements for decisions and loops in programming languages like C and Java, the bedrock statement that implements them at the instruction set level is the conditional branch.

Check Yourself

I. C has many statements for decisions and loops, while LEGv8 has few. Which of the following does or does not explain this imbalance? Why?

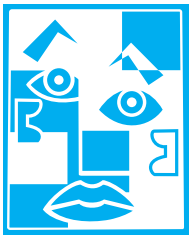
1. More decision statements make code easier to read and understand.
2. Fewer decision statements simplify the task of the underlying layer that is responsible for execution.
3. More decision statements mean fewer lines of code, which generally reduces coding time.
4. More decision statements mean fewer lines of code, which generally results in the execution of fewer operations.

II. Why does C provide two sets of operators for AND (& and &&) and two sets of operators for OR (| and ||), while LEGv8 doesn't?

1. Logical operations AND and ORR implement & and |, while conditional branches implement && and ||.
2. The previous statement has it backwards: && and || correspond to logical operations, while & and | map to conditional branches.
3. They are redundant and mean the same thing: && and || are simply inherited from the programming language B, the predecessor of C.

2.8 Supporting Procedures in Computer Hardware

procedure A stored subroutine that performs a specific task based on the parameters with which it is provided.



A **procedure** or function is one tool programmers use to structure programs, both to make them easier to understand and to allow code to be reused. Procedures allow the programmer to concentrate on just one portion of the task at a time; parameters act as an interface between the procedure and the rest of the program and data, since they can pass values and return results. We describe the equivalent to procedures in Java in Section 2.15, but Java needs everything from a computer that C needs. Procedures are one way to implement **abstraction** in software.

You can think of a procedure like a spy who leaves with a secret plan, acquires resources, performs the task, covers his or her tracks, and then returns to the point of origin with the desired result. Nothing else should be perturbed once the mission is complete. Moreover, a spy operates on only a "need to know" basis, so the spy can't make assumptions about the spymaster.

Similarly, in the execution of a procedure, the program must follow these six steps:

1. Put parameters in a place where the procedure can access them.
2. Transfer control to the procedure.
3. Acquire the storage resources needed for the procedure.
4. Perform the desired task.
5. Put the result value in a place where the calling program can access it.
6. Return control to the point of origin, since a procedure can be called from several points in a program.

As mentioned above, registers are the fastest place to hold data in a computer, so we want to use them as much as possible. LEGv8 software follows the following convention for procedure calling in allocating its 32 registers:

- `X0-X7`: eight parameter registers in which to pass parameters or return values.
- `LR (X30)`: one return address register to return to the point of origin.

branch-and-link instruction An instruction that branches to an address and simultaneously saves the address of the following instruction in a register (`LR` or `X30` in LEGv8).

In addition to allocating these registers, LEGv8 assembly language includes an instruction just for the procedures: it branches to an address and simultaneously saves the address of the following instruction in register `LR` (`X30`). The **branch-and-link instruction** (`BL`) is simply written

```
BL ProcedureAddress
```

The *link* portion of the name means that an address or link is formed that points to the calling site to allow the procedure to return to the proper address. This "link," stored in register LR (register 30), is called the **return address**. The return address is needed because the same procedure could be called from several parts of the program.

return address A link to the calling site that allows a procedure to return to the proper address; in LEGv8 it is stored in register LR (X30).

To support the return from a procedure, computers like LEGv8 use the *branch register* instruction (BR), introduced above to help with case statements, meaning an unconditional branch to the address specified in a register:

```
BR    LR
```

The branch register instruction branches to the address stored in register LR—which is just what we want. Thus, the calling program, or **caller**, puts the parameter values in X0–X7 and uses BL X to branch to procedure X (sometimes named the **callee**). The callee then performs the calculations, places the results in the same parameter registers, and returns control to the caller using BR LR.

caller The program that instigates a procedure and provides the necessary parameter values.

callee A procedure that executes a series of stored instructions based on parameters provided by the caller and then returns control to the caller.

Implicit in the stored-program idea is the need to have a register to hold the address of the current instruction being executed. For historical reasons, this register is almost always called the **program counter**, abbreviated *PC* in the LEGv8 architecture, although a more sensible name would have been *instruction address register*. The BL instruction actually saves PC + 4 in register LR to link to the byte address of the following instruction to set up the procedure return.

program counter (PC) The register containing the address of the instruction in the program being executed.

Using More Registers

Suppose a compiler needs more registers for a procedure than the eight argument registers. Since we must cover our tracks after our mission is complete, any registers needed by the caller must be restored to the values that they contained *before* the procedure was invoked. This situation is an example in which we need to spill registers to memory, as mentioned in the *Hardware/Software Interface* section on page 72.

The ideal data structure for spilling registers is a **stack**—a last-in-first-out queue. A stack needs a pointer to the most recently allocated address in the stack to show where the next procedure should place the registers to be spilled or where old register values are found. The **stack pointer** (SP), which is just one of the 32 registers, is adjusted by one doubleword for each register that is saved or restored. Stacks are so popular that they have their own buzzwords for transferring data to and from the stack: placing data onto the stack is called a **push**, and removing data from the stack is called a **pop**.

stack A data structure for spilling registers organized as a last-in-first-out queue.

stack pointer A value denoting the most recently allocated address in a stack that shows where registers should be spilled or where old register values can be found. In LEGv8, it is register SP.

push Add element to stack.

pop Remove element from stack.

By historical precedent, stacks "grow" from higher addresses to lower addresses. This convention means that you push values onto the stack by subtracting from the stack pointer. Adding to the stack pointer shrinks the stack, thereby popping values off the stack.

Elaboration: As mentioned above, in the full ARMv8 instruction set, the stack pointer is folded into register 31. In some instructions—data transfers and arithmetic immediates that don't set flags when it is the destination register or first source register—register 31 indicates SP but in the rest, such as arithmetic register instructions or in flag setting instructions, it indicates the *zero register* (XZR). Given that this trick only saves one register, would complicate the datapath in Chapter 4, and it is a bit confusing, LEGv8 just assumes SP is one of the other 31 general purpose registers; we use X28 for SP.

EXAMPLE

Compiling a C Procedure That Doesn't Call Another Procedure

Let's turn the example on page 66 from Section 2.2 into a C procedure:

```
long long int leaf_example (long long int g, long long
int h, long long int i, long long int j)
{
       long long int f;

        f = (g + h) – (i + j);
        return f;
}
```

What is the compiled LEGv8 assembly code?

ANSWER

The parameter variables g, h, i, and j correspond to the argument registers X0, X1, X2, and X3, and f corresponds to X19. The compiled program starts with the label of the procedure:

```
leaf_example:
```

The next step is to save the registers used by the procedure. The C assignment statement in the procedure body is identical to the example on page 68, which uses two temporary registers (X9 and X10). Thus, we need to save three registers: X19, X9, and X10. We "push" the old values onto the stack by creating space for three doublewords (24 bytes) on the stack and then store them:

```
SUBI SP,  SP, #24      // adjust stack to make room for 3 items
STUR X10, [SP,#16]     // save register X10 for use afterwards
STUR X9,  [SP,#8]      // save register X9 for use afterwards
STUR X19, [SP,#0]      // save register X19 for use afterwards
```

Figure 2.11 shows the stack before, during, and after the procedure call.

The next three statements correspond to the body of the procedure, which follows the example on page 68:

```
ADD X9,X0,X1   // register X9 contains g + h
ADD X10,X2,X3  // register X10 contains i + j
SUB X19,X9,X10 // f = X9 – X10, which is (g + h) – (i + j)
```

To return the value of `f`, we copy it into a parameter register:

```
ADD X0,X19,XZR // returns f (X0 = X19 + 0)
```

Before returning, we restore the three old values of the registers we saved by "popping" them from the stack:

```
LDUR X19, [SP,#0]   // restore register X19 for caller
LDUR X9, [SP,#8]    // restore register X9 for caller
LDUR X10, [SP,#16]  // restore register X10 for caller
ADDI SP,SP,#24      // adjust stack to delete 3 items
```

The procedure ends with a branch register using the return address:

```
BR     LR     // branch back to calling routine
```

In the previous example, we used temporary registers and assumed their old values must be saved and restored. To avoid saving and restoring a register whose value is never used, which might happen with a temporary register, LEGv8 software separates 19 of the registers into two groups:

- `X9-X17`: temporary registers that are *not* preserved by the callee (called procedure) on a procedure call
- `X19-X28`: saved registers that must be preserved on a procedure call (if used, the callee saves and restores them)

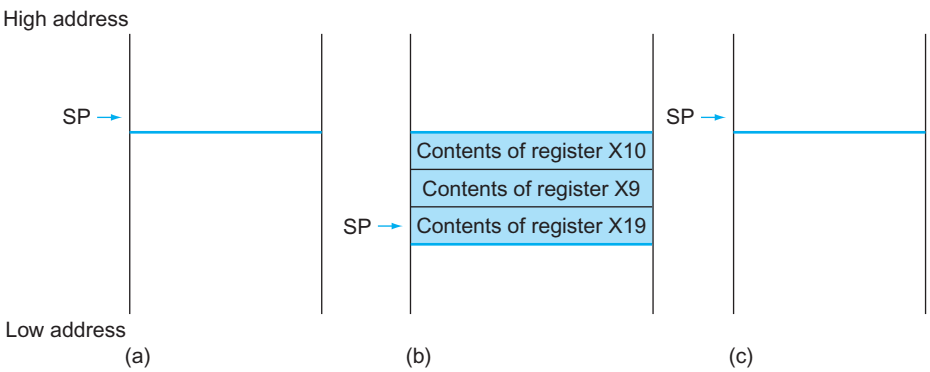


FIGURE 2.11 The values of the stack pointer and the stack (a) before, (b) during, and (c) after the procedure call. The stack pointer always points to the "top" of the stack, or the last doubleword in the stack in this drawing.

This simple convention reduces register spilling. In the example above, since the caller does not expect registers X9 and X10 to be preserved across a procedure call, we can drop two stores and two loads from the code. We still must save and restore X19, since the callee must assume that the caller needs its value.

Nested Procedures

Procedures that do not call others are called *leaf* procedures. Life would be simple if all procedures were leaf procedures, but they aren't. Just as a spy might employ other spies as part of a mission, who in turn might use even more spies, so do procedures invoke other procedures. Moreover, recursive procedures even invoke "clones" of themselves. Just as we need to be careful when using registers in procedures, attention must be paid when invoking nonleaf procedures.

For example, suppose that the main program calls procedure A with an argument of 3, by placing the value 3 into register X0 and then using BL A. Then suppose that procedure A calls procedure B via BL B with an argument of 7, also placed in X0. Since A hasn't finished its task yet, there is a conflict over the use of register X0. Similarly, there is a conflict over the return address in register LR, since it now has the return address for B. Unless we take steps to prevent the problem, this conflict will eliminate procedure A's ability to return to its caller.

One solution is to push all the other registers that must be preserved on the stack, just as we did with the saved registers. The caller pushes any argument registers (X0-X7) or temporary registers (X9-X17) that are needed after the call. The callee pushes the return address register LR and any saved registers (X19-X25) used by the callee. The stack pointer SP is adjusted to account for the number of registers placed on the stack. Upon the return, the registers are restored from memory, and the stack pointer is readjusted.

EXAMPLE

Compiling a Recursive C Procedure, Showing Nested Procedure Linking

Let's tackle a recursive procedure that calculates factorial:

```
long long int fact (long long int n)
{
        if (n < 1) return (1);
                else return (n * fact(n – 1));
}
```

What is the LEGv8 assembly code?

ANSWER

The parameter variable `n` corresponds to the argument register `X0`. The compiled program starts with the label of the procedure and then saves two registers on the stack, the return address and `X0`:

```
fact:
    SUBI  SP, SP, #16 // adjust stack for 2 items
    STUR  LR, [SP,#8] // save the return address
    STUR  X0, [SP,#0] // save the argument n
```

The first time `fact` is called, `STUR` saves an address in the program that called `fact`. The next two instructions test whether `n` is less than 1, going to `L1` if `n ≥ 1`.

```
SUBIS    ZXR,X0, #1  // test for n < 1
B.GE     L1          // if n >= 1, go to L1
```

If `n` is less than 1, `fact` returns 1 by putting 1 into a value register: it adds 1 to 0 and places that sum in `X1`. It then pops the two saved values off the stack and branches to the return address:

```
ADDI    X1,XZR, #1  // return 1
ADDI    SP,SP,#16   // pop 2 items off stack
BR      LR          // return to caller
```

Before popping two items off the stack, we could have loaded `X0` and `LR`. Since `X0` and `LR` don't change when `n` is less than 1, we skip those instructions.

If `n` is not less than 1, the argument `n` is decremented and then `fact` is called again with the decremented value:

```
L1: SUBI X0,X0,#1  // n >= 1: argument gets (n – 1)
    BL fact        // call fact with (n – 1)
```

The next instruction is where `fact` returns. Now the old return address and old argument are restored, along with the stack pointer:

```
LDUR   X0, [SP,#0]   // return from BL: restore argument n
LDUR   LR, [SP,#8]   // restore the return address
ADDI   SP, SP, #16   // adjust stack pointer to pop 2 items
```

Next, the value register `X1` gets the product of old argument `X0` and the current value of the value register. We assume a multiply instruction is available, even though it is not covered until Chapter 3:

```
MUL     X1,X0,X1     // return n * fact (n – 1)
```

Finally, `fact` branches again to the return address:

```
BR     LR     // return to the caller
```

Hardware/ Software Interface

A C variable is generally a location in storage, and its interpretation depends both on its *type* and *storage class*. Example types include integers and characters (see Section 2.9). C has two storage classes: *automatic* and *static*. Automatic variables are local to a procedure and are discarded when the procedure exits. Static variables exist across exits from and entries to procedures. C variables declared outside all procedures are considered static, as are any variables declared using the keyword *static*. The rest are automatic. To simplify access to static data, some LEGv8 compilers reserve a register, called the **global pointer**, or `GP`. For example `X27` could be reserved for `GP`.

global pointer The register that is reserved to point to the static area.

Figure 2.12 summarizes what is preserved across a procedure call. Note that several schemes preserve the stack, guaranteeing that the caller will get the same data back on a load from the stack as it stored onto the stack. The stack above `SP` is preserved simply by making sure the callee does not write above `SP`; `SP` is itself preserved by the callee adding exactly the same amount that was subtracted from it; and the other registers are preserved by saving them on the stack (if they are used) and restoring them from there.

Preserved	Not preserved
Saved registers: `X19-X27`	Temporary registers: `X9-X15`
Stack pointer register: `X28(SP)`	Argument/Result registers: `X0-X7`
Frame pointer register: `X29(FP)`	
Link Register (return address): `X30(LR)`	
Stack above the stack pointer	Stack below the stack pointer

FIGURE 2.12 What is and what is not preserved across a procedure call. If the software relies on the global pointer register, discussed in the following subsections, it is also preserved.

procedure frame Also called **activation record**. The segment of the stack containing a procedure's saved registers and local variables.

frame pointer A value denoting the location of the saved registers and local variables for a given procedure.

Allocating Space for New Data on the Stack

The final complexity is that the stack is also used to store variables that are local to the procedure but do not fit in registers, such as local arrays or structures. The segment of the stack containing a procedure's saved registers and local variables is called a **procedure frame** or **activation record**. Figure 2.13 shows the state of the stack before, during, and after the procedure call.

Some ARMv8 compilers use a **frame pointer** (`FP`) to point to the first doubleword of the frame of a procedure. A stack pointer might change during the procedure, and so references to a local variable in memory might have different offsets depending on where they are in the procedure, making the procedure harder to understand. Alternatively, a frame pointer offers a stable base register within a procedure for local memory-references. Note that an activation record appears on the stack whether or not an explicit frame pointer is used. We've been avoiding using `FP` by avoiding changes to `SP` within a procedure: in our examples, the stack is adjusted only on entry to and exit from the procedure.

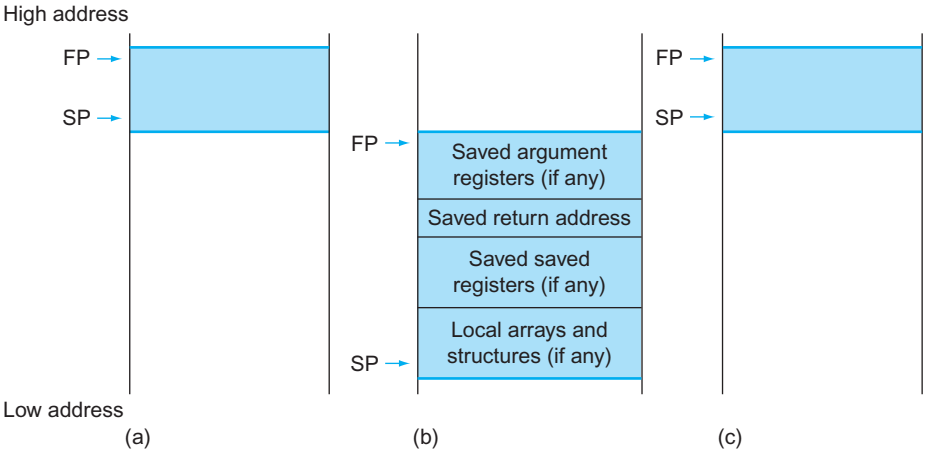


FIGURE 2.13 Illustration of the stack allocation (a) before, (b) during, and (c) after the procedure call. The frame pointer (`FP` or `X29`) points to the first doubleword of the frame, often a saved argument register, and the stack pointer (`SP`) points to the top of the stack. The stack is adjusted to make room for all the saved registers and any memory-resident local variables. Since the stack pointer may change during program execution, it's easier for programmers to reference variables via the stable frame pointer, although it could be done just with the stack pointer and a little address arithmetic. If there are no local variables on the stack within a procedure, the compiler will save time by *not* setting and restoring the frame pointer. When a frame pointer is used, it is initialized using the address in `SP` on a call, and `SP` is restored using `FP`. This information is also found in Column 4 of the LEGv8 Reference Data Card at the front of this book.

Allocating Space for New Data on the Heap

In addition to automatic variables that are local to procedures, C programmers need space in memory for static variables and for dynamic data structures. Figure 2.14 shows the LEGv8 convention for allocation of memory when running the Linux operating system. The stack starts in the high end of the user addresses space (see Chapter 5) and grows down. The first part of the low end of memory is reserved, followed by the home of the LEGv8 machine code, traditionally called the **text segment**. Above the code is the *static data segment*, which is the place for constants and other static variables. Although arrays tend to be a fixed length and thus are a good match to the static data segment, data structures like linked lists tend to grow and shrink during their lifetimes. The segment for such data structures is traditionally called the *heap*, and it is placed next in memory. Note that this allocation allows the stack and heap to grow toward each other, thereby allowing the efficient use of memory as the two segments wax and wane.

text segment The segment of a UNIX object file that contains the machine language code for routines in the source file.

C allocates and frees space on the heap with explicit functions. `malloc()` allocates space on the heap and returns a pointer to it, and `free()` releases space on the heap to which the pointer points. C programs control memory allocation, which is the source of many common and difficult bugs. Forgetting to free space leads to a "memory leak," which ultimately uses up so much memory that the operating system may crash. Freeing space too early leads to "dangling pointers," which can cause pointers to point to things that the program never intended. Java uses automatic memory allocation and garbage collection just to avoid such bugs.

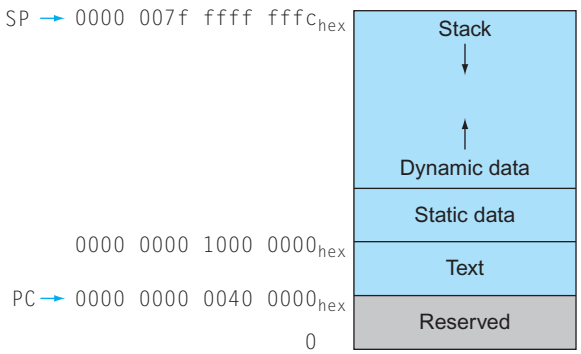


FIGURE 2.14 The LEGv8 memory allocation for program and data. These addresses are only a software convention, and not part of the LEGv8 architecture. The user address space is set to 2^{39} of the potential 2^{64} total address space given a 64-bit architecture (see Chapter 5). The stack pointer is initialized to 0000 007f ffff fffc$_{hex}$ and grows down toward the data segment. At the other end, the program code ("text") starts at 0000 0000 0040 0000$_{hex}$. The static data starts immediately after the end of the text segment; in this example, we assume that address is 0000 0000 1000 0000$_{hex}$. Dynamic data, allocated by `malloc` in C and by `new` in Java, is next. It grows up toward the stack in an area called the *heap*. This information is also found in Column 4 of the LEGv8 Reference Data Card at the front of this book.

COMMON CASE FAST

Figure 2.15 summarizes the register conventions for the LEGv8 assembly language. This convention is another example of making the **common case fast**: most procedures can be satisfied with up to eight argument registers, nine saved registers, and seven temporary registers without ever going to memory.

Elaboration: What if there are more than eight parameters? The LEGv8 convention is to place the extra parameters on the stack just above the frame pointer. The procedure then expects the first eight parameters to be in registers `X0` through `X7` and the rest in memory, addressable via the frame pointer.

As mentioned in the caption of Figure 2.13, the frame pointer is convenient because all references to variables in the stack within a procedure will have the same offset. The frame pointer is not necessary, however. The ARMv8 C compiler uses a frame pointer, but some C compilers do not; they treat register 29 as another save register.

Elaboration: Some recursive procedures can be implemented iteratively without using recursion. Iteration can significantly improve performance by removing the overhead associated with recursive procedure calls. For example, consider a procedure used to accumulate a sum:

```
long long int sum (long long int n, long long int acc) {
     if (n >0)
          return sum(n – 1, acc + n);
     else
          return acc;
}
```

Name	Register number	Usage	Preserved on call?
X0-X7	0–7	Arguments/Results	no
X8	8	Indirect result location register	no
X9-X15	9–15	Temporaries	no
X16 (IP0)	16	May be used by linker as a scratch register; other times used as temporary register	no
X17 (IP1)	17	May be used by linker as a scratch register; other times used as temporary register	no
X18	18	Platform register for platform independent code; otherwise a temporary register	no
X19-X27	19–27	Saved	yes
X28 (SP)	28	Stack Pointer	yes
X29 (FP)	29	Frame Pointer	yes
X30 (LR)	30	Link Register (return address)	yes
XZR	31	The constant value 0	n.a.

FIGURE 2.15 LEGv8 register conventions. This information is also found in Column 2 of the LEGv8 Reference Data Card at the front of this book. X8 is used by procedures that return a result via a pointer. ARM discourages the use of registers X16 to X18 as X16 and X17 may be used by the linker (see Section 2.12), and X18 may be used to create platform independent code, which would be specified by the platforms Application Binary Interface.

Consider the procedure call sum(3,0). This will result in recursive calls to sum(2,3), sum(1,5), and sum(0,6), and then the result 6 will be returned four times. This recursive call of sum is referred to as a *tail call*, and this example use of tail recursion can be implemented very efficiently (assume X0 = n, X1 = acc, and the result goes into X2):

```
sum: SUBS XZR, X0, XZR  // compare n to 0
     B.LE sum_exit      // go to sum_exit if n <= 0
     ADD X1, X1, X0     // add n to acc
     SUBI X0, X0, #1    // subtract 1 from n
     B sum              // go to sum
sum_exit:
     ADD X2, X1, XZR    // return value acc
     BR LR              // return to caller
```

Check Yourself

Which of the following statements about C and Java is generally true?

1. C programmers manage data explicitly, while it's automatic in Java.
2. C leads to more pointer bugs and memory leak bugs than does Java.

!(@|=>(wow open tab at bar is great)

Fourth line of the keyboard poem "Hatless Atlas," 1991 (some give names to ASCII characters: "!" is "wow," "(" is open, "|" is bar, and so on).

2.9 Communicating with People

Computers were invented to crunch numbers, but as soon as they became commercially viable they were used to process text. Most computers today offer 8-bit bytes to represent characters, with the *American Standard Code for Information Interchange* (ASCII) being the representation that nearly everyone follows. Figure 2.16 summarizes ASCII.

ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

FIGURE 2.16 ASCII representation of characters. Note that upper- and lowercase letters differ by exactly 32; this observation can lead to shortcuts in checking or changing upper- and lowercase. Values not shown include formatting characters. For example, 8 represents a backspace, 9 represents a tab character, and 13 a carriage return. Another useful value is 0 for null, the value the programming language C uses to mark the end of a string.

ASCII versus Binary Numbers

EXAMPLE

We could represent numbers as strings of ASCII digits instead of as integers. How much does storage increase if the number 1 billion is represented in ASCII versus a 32-bit integer?

ANSWER

One billion is 1,000,000,000, so it would take 10 ASCII digits, each 8 bits long. Thus the storage expansion would be (10 × 8)/32 or 2.5. Beyond the expansion in storage, the hardware to add, subtract, multiply, and divide such decimal numbers is difficult and would consume more energy. Such difficulties explain why computing professionals are raised to believe that binary is natural and that the occasional decimal computer is bizarre.

A series of instructions can extract a byte from a doubleword, so load register and store register are sufficient for transferring bytes as well as words. Because of the popularity of text in some programs, however, LEGv8 provides instructions to move bytes. *Load byte* (`LDURB`) loads a byte from memory, placing it in the rightmost 8 bits of a register. *Store byte* (`STURB`) takes a byte from the rightmost 8 bits of a register and writes it to memory. Thus, we copy a byte with the sequence

```
LDURB X9,[X0,#0]    // Read byte from source
STURB X9,[X1,#0]    // Write byte to destination
```

Characters are normally combined into strings, which have a variable number of characters. There are three choices for representing a string: (1) the first position of the string is reserved to give the length of a string, (2) an accompanying variable has the length of the string (as in a structure), or (3) the last position of a string is indicated by a character used to mark the end of a string. C uses the third choice, terminating a string with a byte whose value is 0 (named null in ASCII). Thus, the string "Cal" is represented in C by the following 4 bytes, shown as decimal numbers: 67, 97, 108, and 0. (As we shall see, Java uses the first option.)